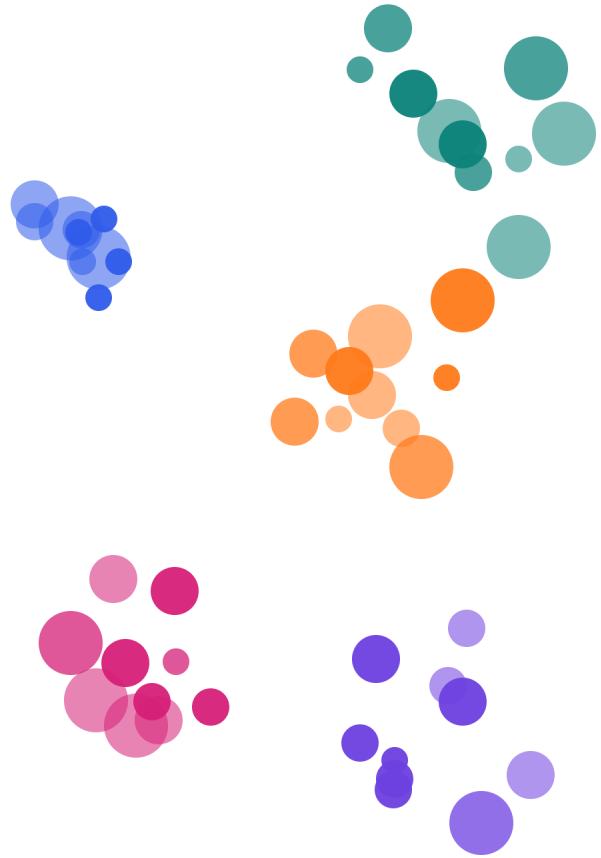
When Your Embeddings Lie

What “similar” really means in embeddings and vector databases

Namita Achyuthan



THE HAPPY PATH

Six simple sentences

I like apples

I like oranges

I like mango

The sky is blue

The sky has clouds

The sky is dark

Two topics. Will the embedding model see that?

THE HAPPY PATH

Similar meaning, nearby vectors



BUT THEN

Now try a fact

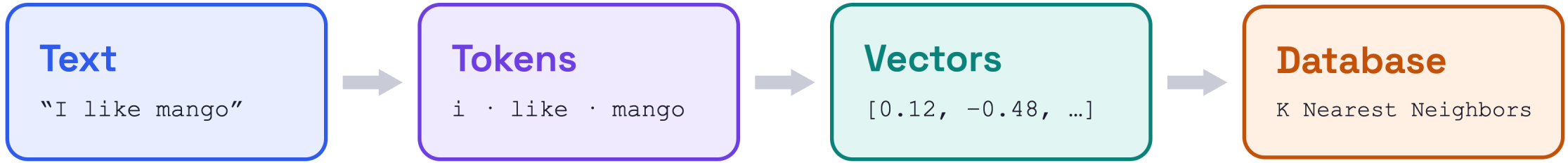
- ✓ The capital of India is Delhi
- ✗ The capital of India is Paris
- ✗ The capital of India is London

The model can’t tell which is true. By the end, you’ll know why.

THE PLAN

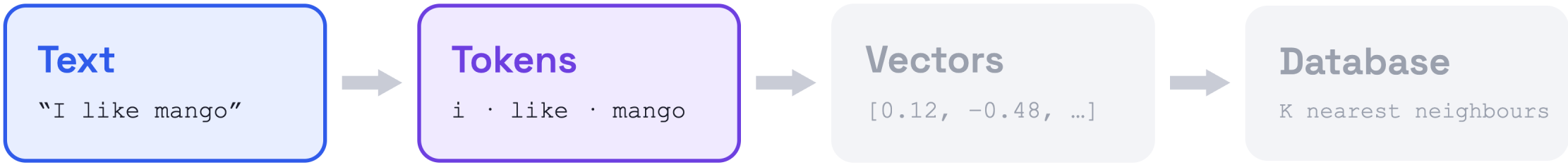
Follow the data

three stages, then we break each one



STAGE 1

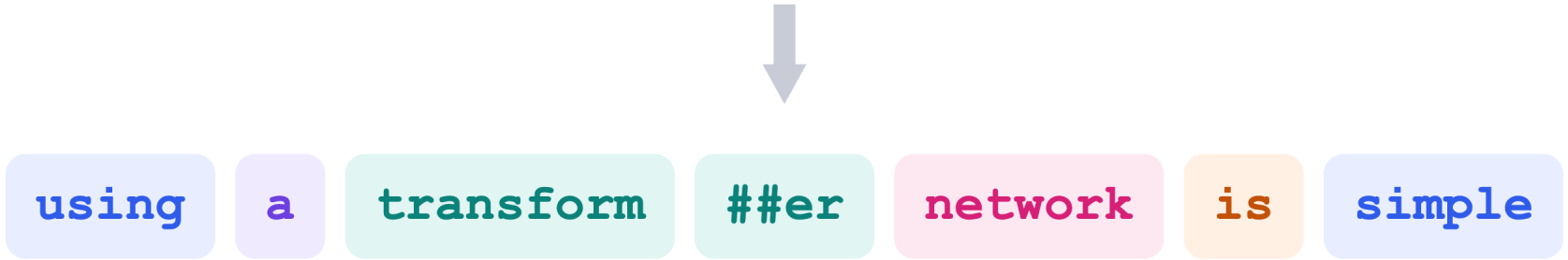
Tokenizing



STAGE 1 · TOKENIZING

Step 1: split text into tokens

“Using a Transformer network is simple”



common words stay whole, while rare words split into known pieces

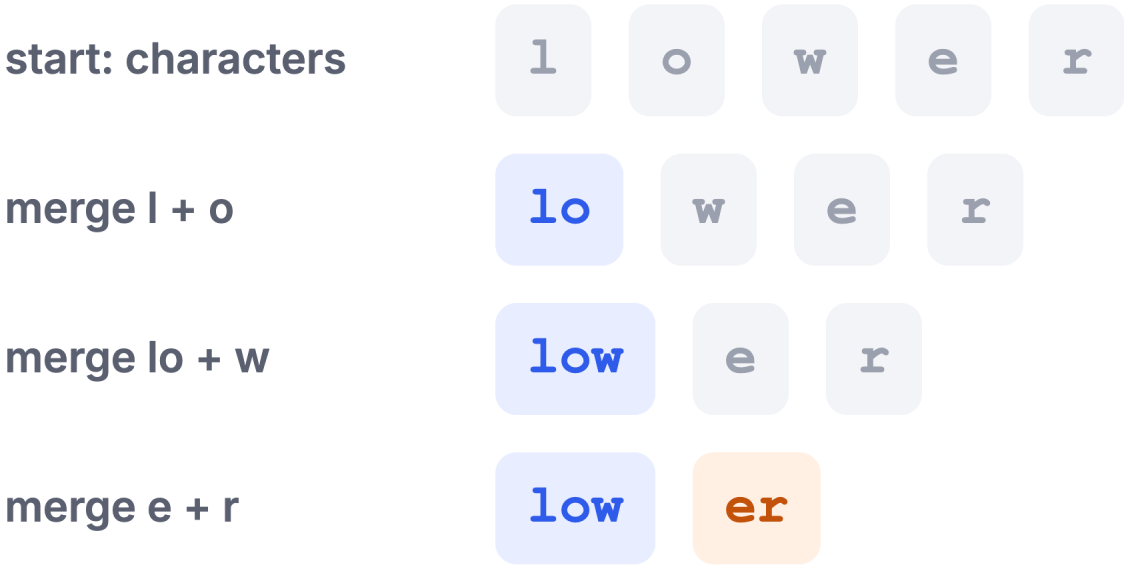
STAGE 1 · TOKENIZING

Four ways to learn a vocabulary

BPE	merge the most frequent pair, repeat	the original subword method
WordPiece	merge the pair that best improves likelihood	BERT
Unigram	start big, prune the least useful pieces	T5, XLM-R (SentencePiece)
Byte-level BPE	BPE on raw bytes, so nothing is unknown	GPT-2, RoBERTa

STAGE 1 · TOKENIZING

BPE: merge the most frequent pair



Frequent words become one token. Rare words stay in pieces.

STAGE 1 · TOKENIZING

WordPiece: merge what belongs together

BPE asks

which pair appears most often?

WordPiece asks

which pair appears together more than chance predicts?

`tokenization` → `token` `##ization`

STAGE 1 · TOKENIZING

Unigram: start big, then prune

huge starting vocabulary	`man` `mango` `ngo` `go` `an` `m`
prune the least useful	`mango` `go` `m`
SentencePiece view	`_I` `_like` `_mango`

Spaces are just another symbol (_). No assumption that spaces separate words.

STAGE 1 · TOKENIZING

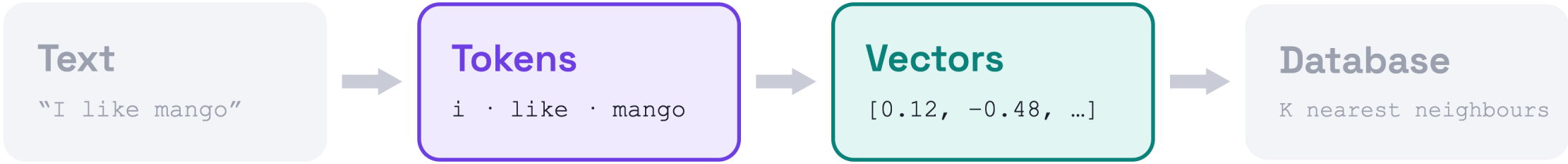
Byte-level: never unknown, not always cheap

a	é	म	ಮ
English letter	accented letter	Devanagari	Kannada
1 byte	2 bytes	3 bytes	3 bytes

If the vocabulary has few merges for a script, each character can cost several tokens

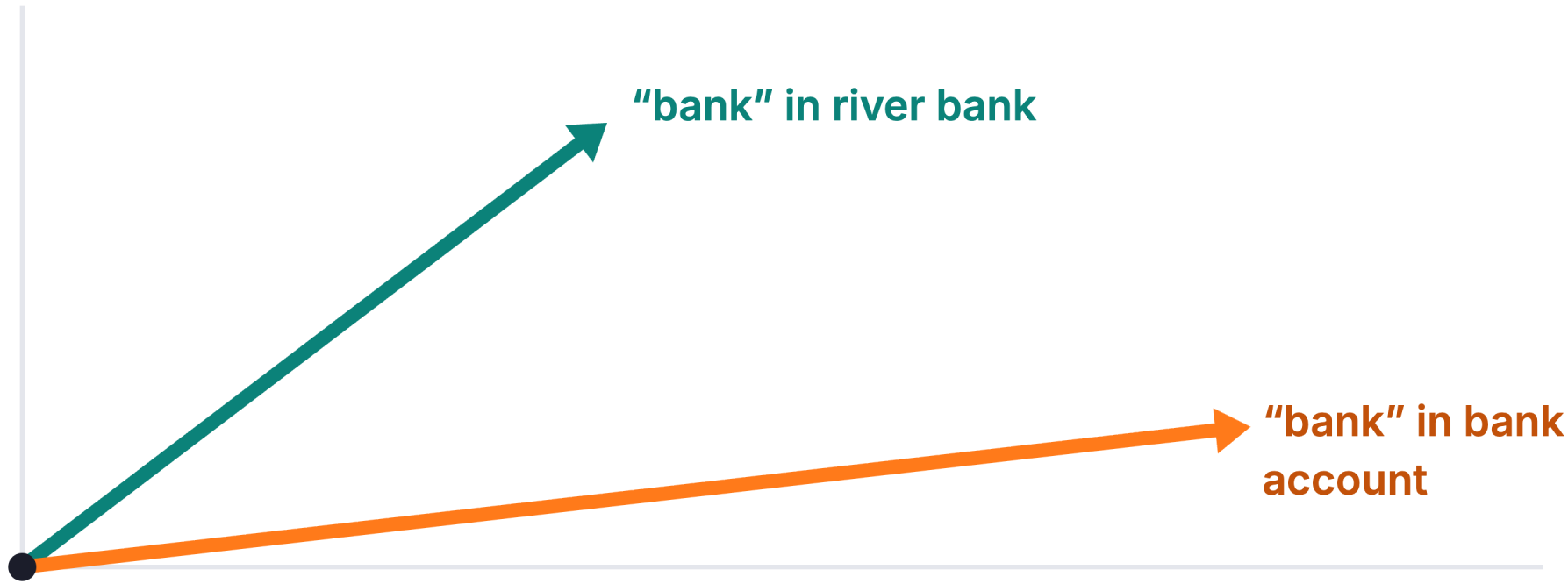
STAGE 2

Embedding



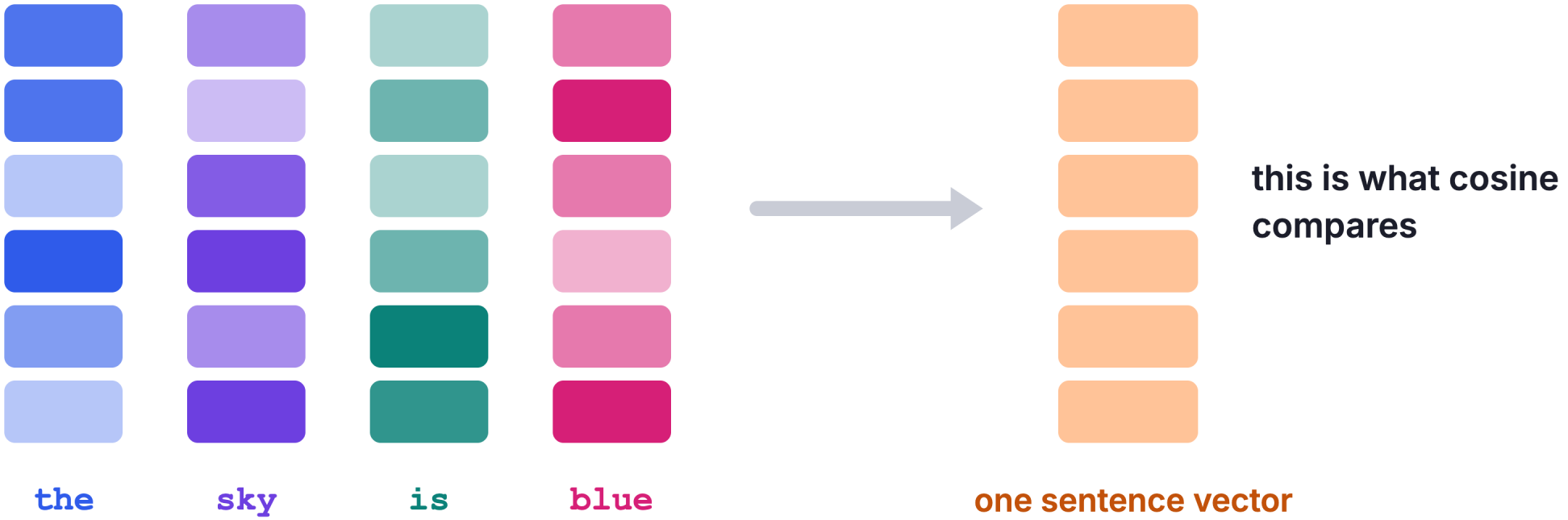
STAGE 2 · EMBEDDING

Step 2: tokens become vectors, shaped by context



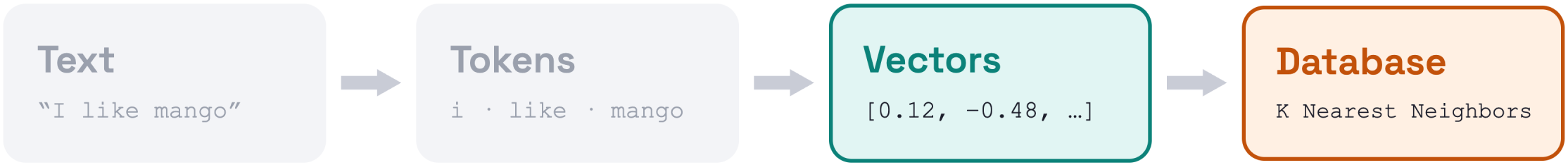
STAGE 2 · EMBEDDING

Step 3: pool into one sentence vector



STAGE 3

Vector database



STAGE 3 · VECTOR DATABASE

Checking every vector doesn’t scale

Exact search

compares every vector

perfect recall

slow at scale

Approximate (ANN)

compares a smart subset

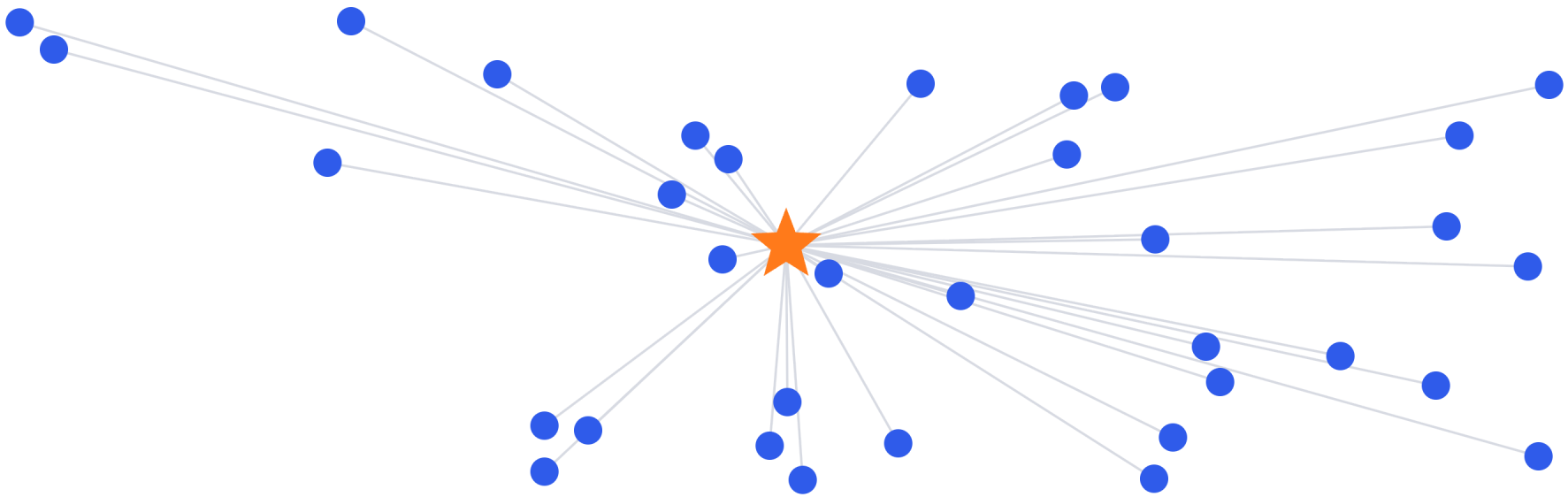
fast

may miss a few true neighbours

Next: four ways to build an index

STAGE 3 · INDEX 1

Flat: compare against everything



Use when small data, or as the answer key

You tune nothing; it’s exact

STAGE 3 · INDEX 2

IVF: cluster first, search nearby

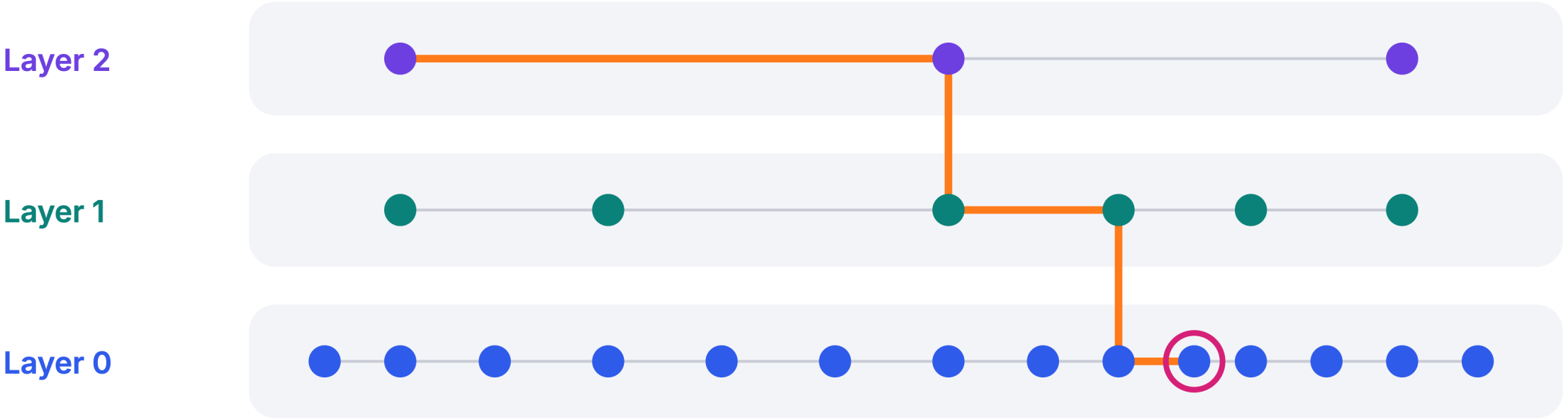


Use when large data, tunable speed vs recall

You tune nprobe: how many clusters to search

STAGE 3 · INDEX 3

HNSW: hop through a layered graph

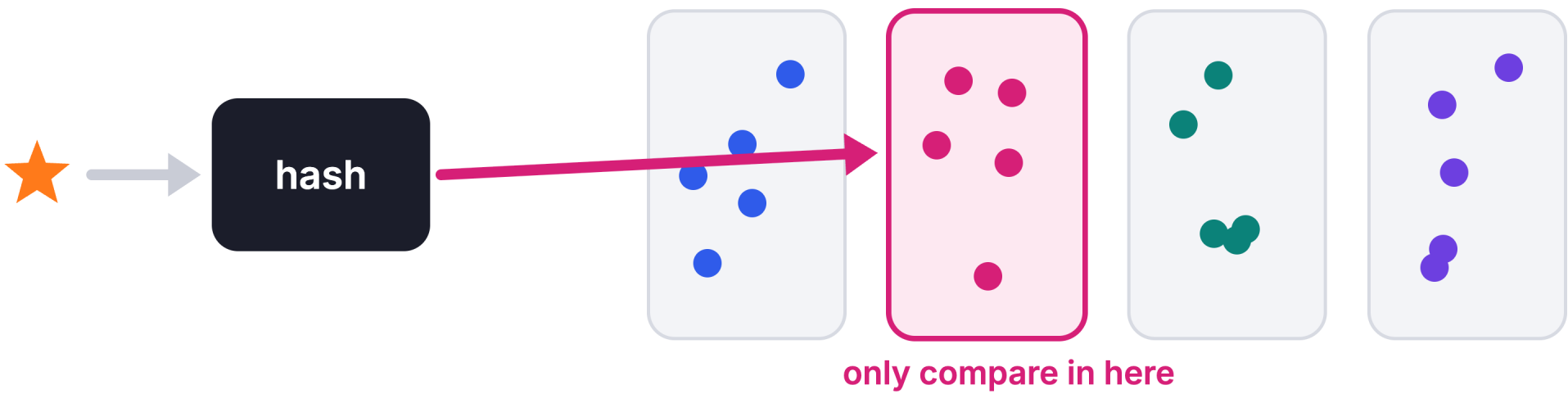


Use when you need speed and recall, and have memory

You tune efSearch: how widely to explore

STAGE 3 · INDEX 4

LSH: hash similar items together

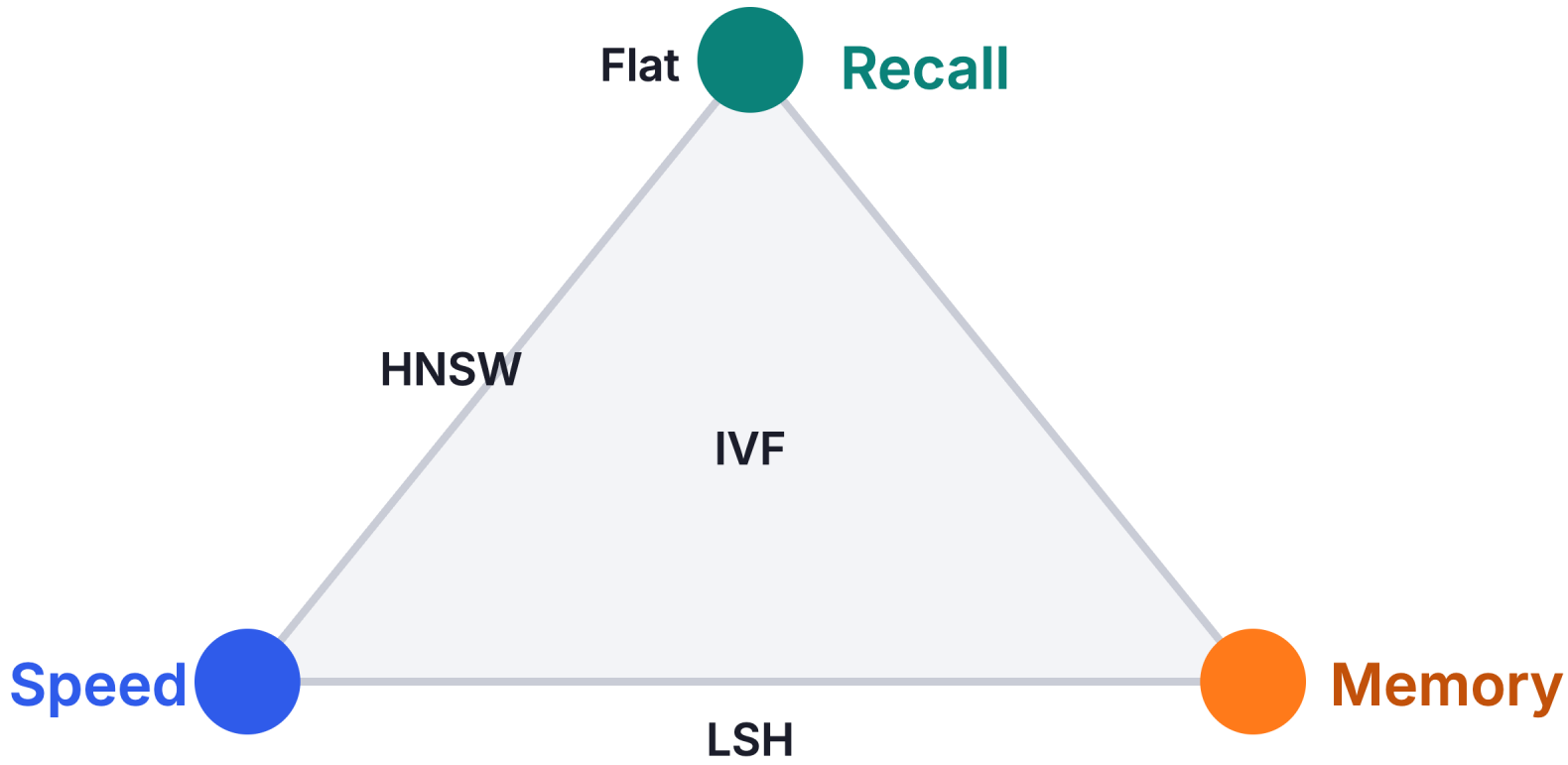


Use when finding near-duplicates, cheaply

You tune number of hash tables and bits

STAGE 3 · VECTOR DATABASE

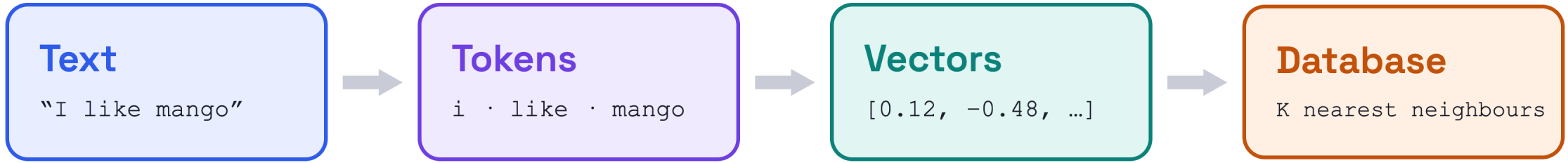
Every index is a trade-off



PART TWO

Now let’s break it

stage by stage



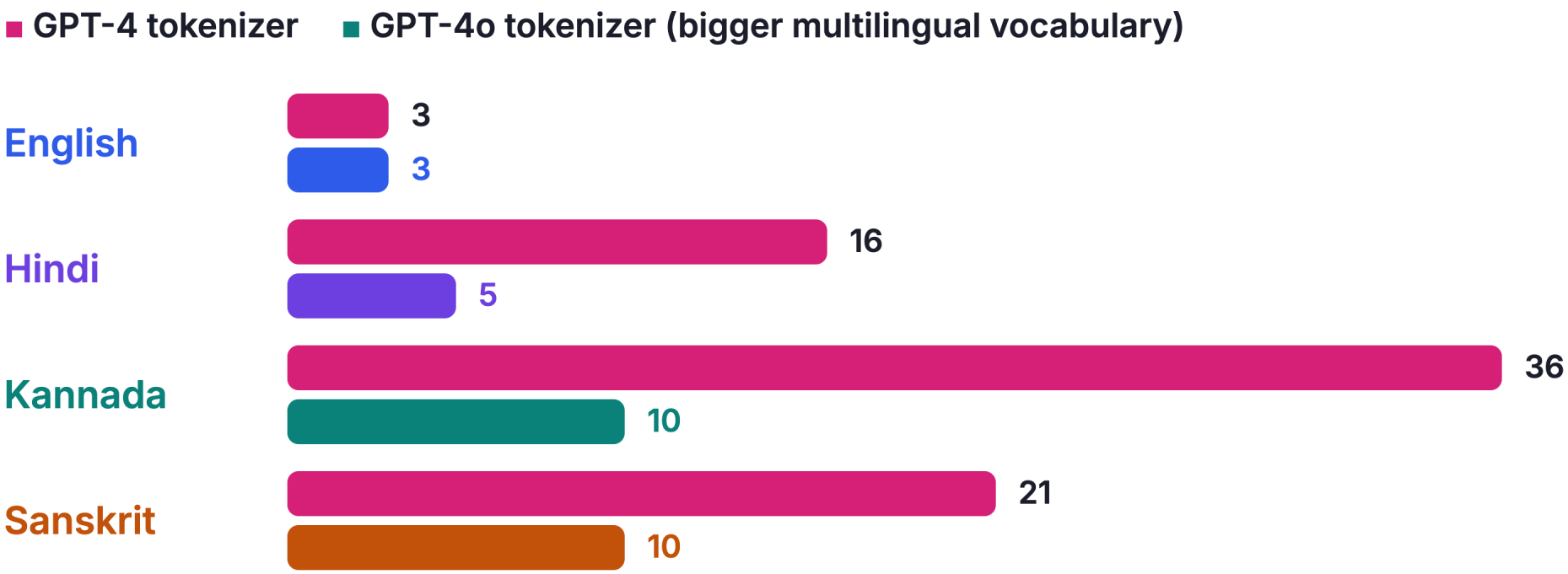
TOKENIZER FAILURES

Same sentence, four languages

English	I like mango
Hindi	मुझे आम पसंद है
Kannada	ನನಗೆ ಮಾವಿನ ಹಣ್ಣು ಇಷ್ಟ
Sanskrit	मह्यम् आम्रफलं रोचते

TOKENIZER FAILURES

“I like mango”: same sentence, different token counts



Counted with OpenAI’s cl100k_base (GPT-4) and o200k_base (GPT-4o) tokenizers

TOKENIZER FAILURES

Why token count matters

Cost

APIs charge per token

Context

fewer words fit in the window

Speed

more tokens, more compute

Quality

fragments carry less meaning

TOKENIZER FAILURES

The right model finds more

मह्यम् आम्रफलं रोचते → I like mango?

English-only model

barely saw Sanskrit in training

sees the query as unfamiliar fragments

Multilingual model

trained on translations across 50+ languages

places both sentences close together

The wrong tokenizer quietly lowers every result after it

TOKENIZER FAILURES

What if the data isn't a language at all?

192.168.1.1

192 . 168 . 1 . 1

IP addresses become number fragments · very different records can score 0.99

A tokenizer built for words turns identifiers into noise

A tokenizer only knows what it was trained on.

EMBEDDING FAILURES

Back to the capitals

- ✓ The capital of India is Delhi
- ✗ The capital of India is Paris
- ✗ The capital of India is London

similarity between them

very high

EMBEDDING FAILURES

Opposites look alike

I love this movie	≈	I don’t love this movie
I like mango	≈	I don’t like mango anymore
The patient has diabetes	≈	The patient does not have diabetes

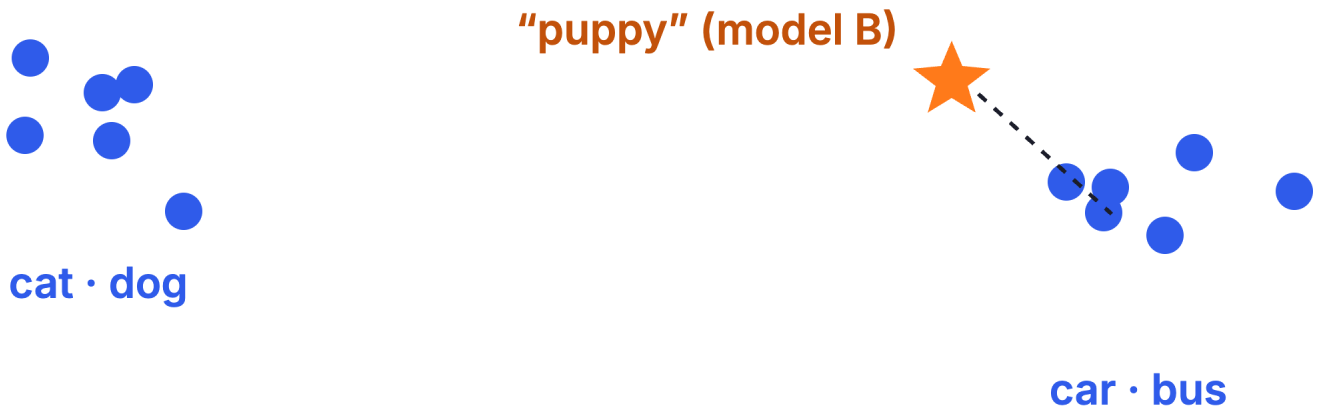
One small word flips the meaning, but barely moves the vector

Embeddings measure what it’s about,

not whether it’s true.

DATABASE FAILURES · 1

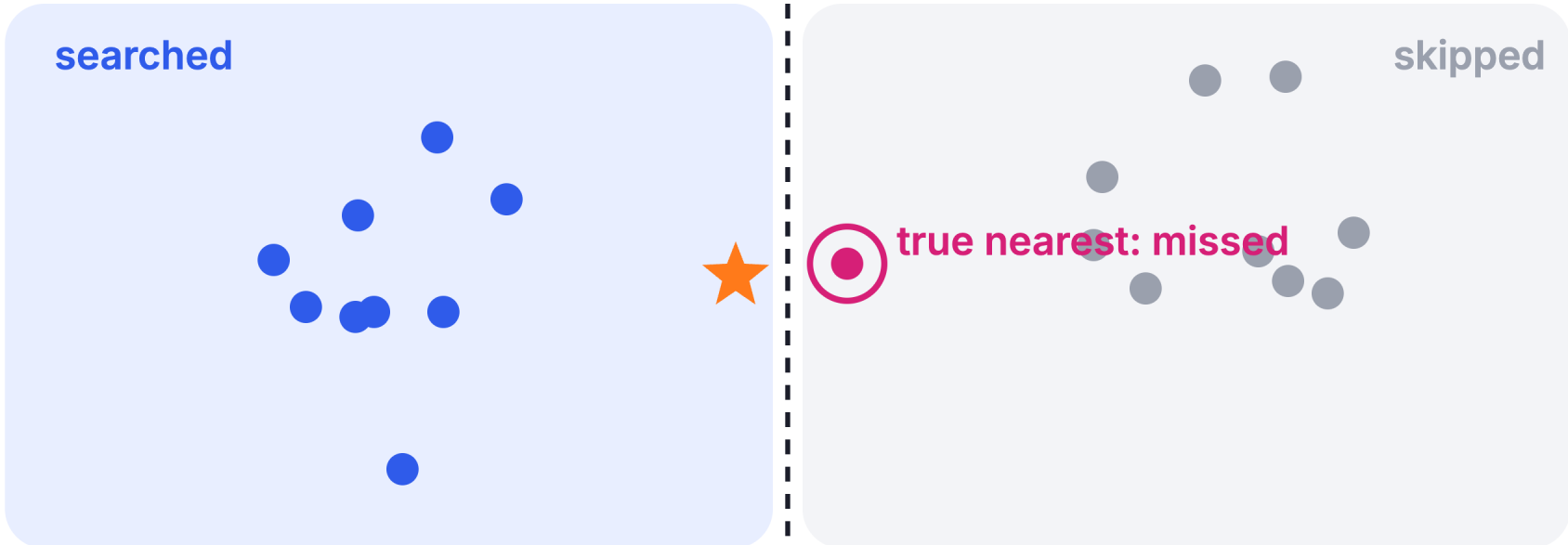
Mixing embedding models



Each model has its own space. Change the model, re-embed everything.

DATABASE FAILURES · 2

The neighbour across the line



Approximate means some true matches are never checked.

DATABASE FAILURES · 3

Filters after the search

Ask for the top 10, then filter to “year = 2026”



Asked for 10, got 2

DATABASE FAILURES · 4

One chunk, two topics



Too big: meaning averages out. Too small: context disappears.

DATABASE FAILURES · 5

The longest vector wins

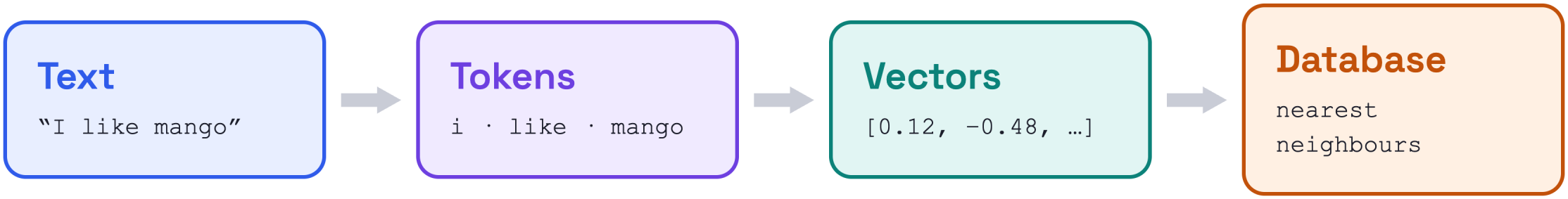


Unnormalized vectors + dot product: length beats meaning

PART THREE

Now let’s fix it

stage by stage



FIXING IT · TOKENIZER

Tokenizer: pick one that knows your data

Match the language

multilingual or Indic models like MuRIL, IndicBERT

Count tokens first

measure on your real data before you choose

Don’t force it into text

IDs, numbers and logs need their own features

Adapt the vocabulary

add domain tokens and fine-tune where it matters

FIXING IT · EMBEDDING

Embeddings: find, don’t verify

Hybrid search

add keyword matching so “Delhi” counts

Reranking

a second model reads query and result together

Trusted sources

retrieve only from data you trust

Hard negatives

fine-tune on near-identical pairs that differ

Use embeddings to find candidates, not to verify them

FIXING IT · VECTOR DATABASE

Database: one fix per failure

Mixed models	change the model, re-embed everything
Missed neighbours	measure recall against brute force, tune nprobe or efSearch
Filters	filter during the search, or fetch more candidates
Chunking	test chunk sizes on real queries
Longest wins	normalize vectors, match the model’s metric

WHAT YOUR CHOICES COST

Bigger embeddings, bigger storage

1.5 GB
384 dimensions

3.1 GB
768 dimensions

6.1 GB
1536 dimensions

One million vectors, float32, before any index overhead

WHAT YOUR CHOICES COST

Same data, four indexes

Index	Recall@10	ms / query	Memory
Flat	1.00	2.4	51 MB
IVF (nprobe 4)	1.00	0.11	52 MB
HNSW (ef 64)	0.98	0.10	78 MB
IVF-PQ (nprobe 16)	0.29	0.12	2.7 MB

100,000 synthetic 128-d vectors on a CPU · notebook section 7 · rerun for your hardware

WHAT YOUR CHOICES COST

Fewer bits, less memory

1×

float32 · 4 bytes per number

4× smaller

int8 · 1 byte per number

32× smaller

binary · 1 bit per number

The price is some recall, so measure it

WHAT YOUR CHOICES COST

The metric decides what “close” means

Cosine

direction only · ignores length

Dot product

direction and length · longer wins

Euclidean

straight-line distance · size matters

A record with 1000× the size can still score cosine = 1.0

WHAT YOUR CHOICES COST

Where should the vectors live?

Library

FAISS

fastest to start, you handle storage and updates

Vector database

Chroma, Milvus

persistence, filtering, scaling

Extension

pgvector

adds vectors to Postgres

Multi-model db

Couchbase

vector, keyword and filtered search beside your app data

WHAT YOUR CHOICES COST

Ask before you choose

- What language or domain is the data?
- What's the memory budget?
- What latency do users need?
- How much data, growing how fast?
- How often does it change?
- Do you need filters?

I work on agent memory.

Everything in this talk is part of the job.

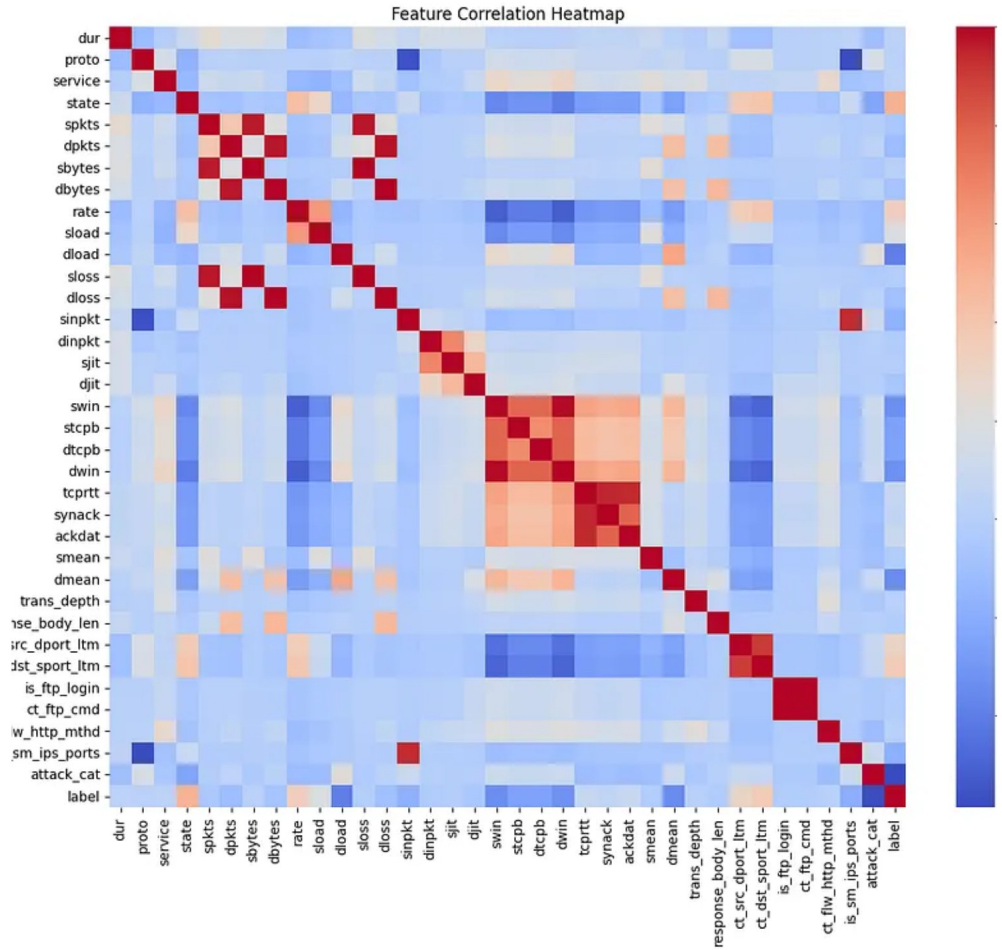
WHY IT MATTERS

Simple in a demo, hard in production

IN THIS TALK	IN AGENT MEMORY
Delhi, Paris and London look alike	“Lives in Delhi” vs “moved to London”: which is current?
“I like mango” ≈ “I like apples”	“Likes mango” vs “doesn’t like mango anymore”
Many tokens squeezed into one vector	The exact date or name gets averaged away

The things that look simple are where production work happens.

Want to see these choices play out on real network data?



WRAPPING UP

Four things to take home

1. Embeddings measure similarity, not truth
2. Tokenizers only know their training data
3. Every index is a trade-off
4. Choose for your constraints, measure your task

Questions?

Namita Achyuthan

